**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное автономное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»**
**(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ **ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ**

КАФЕДРА **КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)**

НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.01 Информатика и вычислительная техника**

# ОТЧЕТ

## по лабораторной работе № 10

## Дисциплина: Языки интернет программирования

Студент ИУ6-64Б Д. О. Романов
(Группа) (Подпись, дата) (И.О. Фамилия)

Преподаватель В. И. Кузнецов
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

## 1. ЦЕЛЬ РАБОТЫ

Цель работы – изучить работу с моделями данных Django ORM, создать модель Product, научиться использовать ModelForm для создания и валидации форм, освоить вывод данных из базы данных в шаблоны и настройку маршрутизации.

## 2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Django ORM представляет собой мощный инструмент объектно-реляционного отображения, который позволяет работать с базой данных через классы Python. Модель в Django – это класс, наследующий django.db.models.Model, где каждый атрибут класса описывает поле таблицы базы данных. Django предоставляет разнообразные типы полей: CharField для строк ограниченной длины, IntegerField для целых чисел, BooleanField для логических значений, а также поддерживает choices для ограничения набора допустимых значений. После определения моделей необходимо создать и применить миграции командами makemigrations и migrate, чтобы Django автоматически сгенерировал соответствующую схему таблиц в базе данных. ModelForm – это специальный класс формы в Django, привязанный к конкретной модели. В отличие от обычной Form, ModelForm автоматически генерирует поля формы на основе определения модели, что значительно сокращает объём кода и устраняет дублирование. ModelForm наследует всю встроенную валидацию типов данных из полей модели, а также позволяет расширять проверку через методы clean_имя_поля и clean(). После успешной валидации данные сохраняются в базу одним вызовом form.save(), при этом Django самостоятельно создаёт объект модели, заполняет его и выполняет INSERT или UPDATE запрос. Менеджер объектов objects является основным интерфейсом для выполнения запросов к базе данных в Django. Через Product.objects.all() можно получить все записи

таблицы, objects.filter(условие) – отфильтровать записи по заданным критериям, objects.create(данные) – создать новую запись напрямую без промежуточного сохранения формы. Менеджер абстрагирует разработчика от написания SQL-запросов вручную и возвращает QuerySet – ленивый объект, который можно дополнительно фильтровать, сортировать и передавать в шаблоны для отображения.

## 3. ХОД ВЫПОЛНЕНИЯ РАБОТЫ

### 3.1. Задание 1. Создание модели Product

Создана модель Product с полями: название (CharField, unique), вес (IntegerField), цена (IntegerField), категория (CharField с choices), наличие аллергенов (BooleanField). Поле name имеет ограничение уникальности. Для категории заданы три варианта: еда, вода, другое. Выполнены миграции: makemigrations и migrate.

**Листинг 1 – models.py**

```
from django.db import models

    class Product(models.Model): cats = [('food', 'еда'),
    ('drink', 'вода'), ('snack', 'другое')]

    name = models.CharField(max_length=200, unique=True,
verbose_name='название')
weight = models.IntegerField(verbose_name='вес в гр')
price = models.IntegerField(verbose_name='цена в руб')
category = models.CharField(max_length=20, choices=cats,
verbose_name='категория')
has_allergens = models.BooleanField(default=False,
verbose_name='Наличие аллергенов')


def __str__(self):
    return self.name
```

### 3.2. Задание 2. Создание формы ModelForm с валидацией

Создан класс ProductForm, наследующий ModelForm. В Meta указана модель Product и список полей. Реализованы методы валидации: clean_name проверяет уникальность названия продукта в базе данных, clean_weight и clean_price проверяют положительность значений.

**Листинг 2 – forms.py**

```
     from django import forms

     from .models import Product

     class  ProductForm(forms.ModelForm):  class  Meta:  model  =
     Product  fields  =  ['name',  'weight',  'price',  'category',
     'has_allergens']

     def clean_name(self):
    name = self.cleaned_data['name']
    if Product.objects.filter(name=name).exists():
        raise forms.ValidationError('продукт уже существует')
    return name

def clean_weight(self):
    weight = self.cleaned_data['weight']
    if weight <= 0:
                    raise  forms.ValidationError('вес  должен  быть
положительным')
    return weight

def clean_price(self):
    price = self.cleaned_data['price']
    if price <= 0:
                   raise  forms.ValidationError('цена  должна  быть
положительной')
    return price
```

### 3.3. Задание 3. Создание представлений

Функция product_list извлекает все объекты Product через Product.objects.all() и передаёт их в шаблон temp_prod.html. Функция product_add обрабатывает GET и POST запросы: при GET создаёт пустую форму, при POST проверяет валидность, сохраняет продукт и выполняет редирект на список.

**Листинг 3 – views.py**

```
from django.shortcuts import render, redirect

from .models import Product

from .forms import ProductForm

     def product_list(request): products = Product.objects.all()
     return render(request, 'products/temp_prod.html',
     {'products': products})

     def product_add(request): if request.method == 'POST': form =
ProductForm(request.POST) if form.is_valid(): form.save() return
redirect('product_list') else: form = ProductForm() return
render(request, 'products/temp_add.html', {'form': form})
```

### 3.4. Задание 4. Настройка маршрутизации

Маршрутизация реализована на двух уровнях. Головной urls.py проекта включает маршруты приложения products по префиксу product/. В urls.py приложения определены два маршрута: корневой – список продуктов, add/ – форма добавления.

**Листинг 3 – views.py**

```
from django.shortcuts import render, redirect

from .models import Product

from .forms import ProductForm

    def product_list(request): products = Product.objects.all()
    return render(request, 'products/temp_prod.html',
    {'products': products})

    def product_add(request): if request.method == 'POST': form =
ProductForm(request.POST) if form.is_valid(): form.save() return
redirect('product_list') else: form = ProductForm() return
render(request, 'products/temp_add.html', {'form': form})
```

**Листинг 4 – urls.py приложения**

```
from django.urls import path

from . import views

    urlpatterns = [ path('', views.product_list,
    name='product_list'), path('add/', views.product_add,
    name='product_add'),]
```

| **Листинг 5 – urls.py проекта** |
| --- |
| `from django.contrib import admin`<br><br>`from django.urls import path, include`<br><br>`urlpatterns = [ path('admin/', admin.site.urls), path('product/', include('products.urls')),]` |

### 3.5. Задание 5. Создание шаблонов

Шаблон temp_prod.html отображает таблицу со всеми продуктами, используя цикл {% for %}. Для категории применяется метод get_category_display для отображения человекочитаемого значения. Для аллергенов используется условный тег {% if %}. Шаблон temp_add.html выводит форму через {{ form.as_p }}.

**Листинг 6 – фрагмент temp_prod.html**

```
<!DOCTYPE html>
<body>
    <h1 style="text-align: center;">список</h1>
    <table>
        <tr>
            <th>название</th>
            <th>вес в гр</th>
            <th>цена в руб</th>
            <th>категория</th>
            <th>аллергены</th>
        </tr>
        {% for product in products %}
        <tr>
            <td>{{ product.name }}</td>
            <td>{{ product.weight }}</td>
            <td>{{ product.price }}</td>
            <td>{{ product.get_category_display }}</td>
            <td>{% if product.has_allergens %}да{% else %}нет{% end
        </tr>
        {% empty %}
        <tr>
            <td style="text-align: center;">продуктов нет</td>
        </tr>
        {% endfor %}
    </table>
    <p style="text-align: center;">
        <a href="{% url 'product_add' %}">добавить продукт</a>
    </p>
</body>
</html>
```

**Листинг 7 – фрагмент temp_add.html**

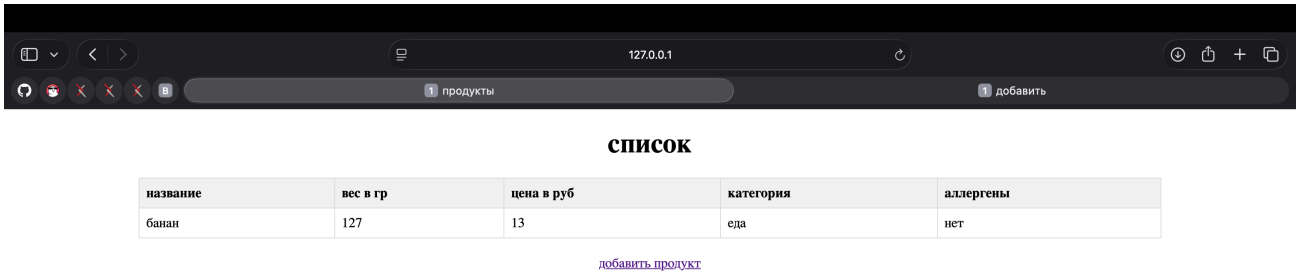
```
<!DOCTYPE html>
<body>
    <h1>добавление продукта</h1>
    <form method="post">
        {% csrf_token %}
        {{ form.non_field_errors }}
        <p>
            <label for="{{ form.name.id_for_label }}">название:</la
            {{ form.name }}
            {{ form.name.errors }}
        </p>
        <p>
            <label for="{{ form.weight.id_for_label }}">вес в гр:</
            {{ form.weight }}
            {{ form.weight.errors }}
        </p>
        <p>
            <label for="{{ form.price.id_for_label }}">цена в руб:<
            {{ form.price }}
            {{ form.price.errors }}
        </p>
        <p>
            <label
   </body>
</html>
```

### 3.6. Проверка работы приложения

Проект запущен командой python manage.py runserver. В консоли виден запуск development-сервера на http://127.0.0.1:8000/. Страница /product/ отображает таблицу с добавленным продуктом «банан». Страница /product/add/ содержит форму с полями модели.

```
lab10 — python ◂ python manage.py runserver — 80×24

Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Us
e a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/e
n/6.0/howto/deployment/
[09/Mar/2026 13:02:27] "GET /product/add/ HTTP/1.1" 200 1725
/Users/dmitryromanov/Desktop/web/web lb10/lab10/products/forms.py changed, reloa
ding.
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
March 09, 2026 - 13:03:22
Django version 6.0.2, using settings 'lab10.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Us
e a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/e
n/6.0/howto/deployment/
```

Рисунок 1 – Консоль сервера Django




**список**

| название | вес в гр | цена в руб | категория | аллергены |
|---|---|---|---|---|
| банан | 127 | 13 | еда | нет |

добавить продукт

Рисунок 2 – Страница списка продуктов /product/

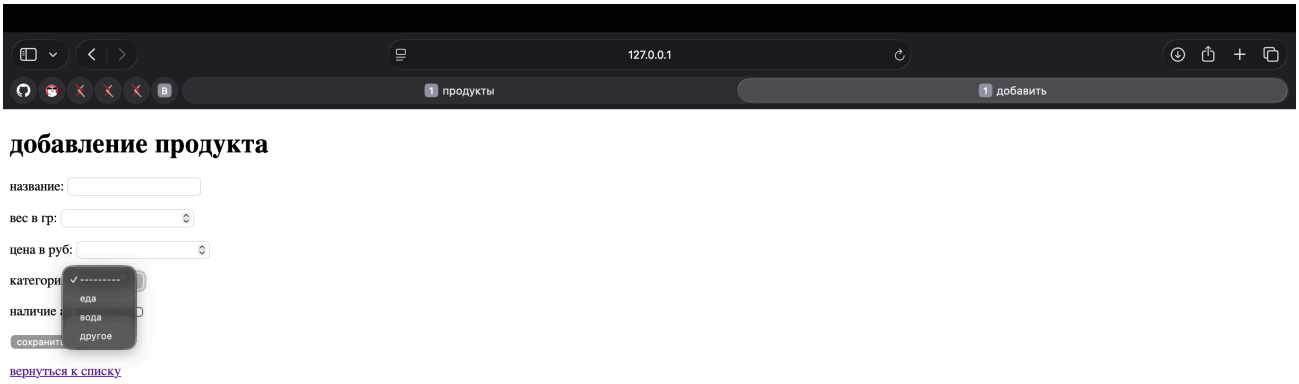



Рисунок 3 – Страница добавления продукта /product/add/

## 4. ВЫВОДЫ

В ходе лабораторной работы были изучены и на практике применены: создание моделей данных Django ORM с различными типами полей, механизм миграций, создание форм ModelForm с привязкой к модели, кастомная валидация данных

через clean-методы, вывод списка объектов из базы данных в шаблон с использованием цикла DTL, двухуровневая маршрутизация URL. Поставленные задачи выполнены полностью.

## 5. ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

**Вопрос 1: Что такое модель в Django ORM?**

Ответ: Модель – это класс Python, наследующий django.db.models.Model, который описывает структуру таблицы в базе данных. Каждый атрибут класса соответствует полю таблицы. Django автоматически создаёт таблицы по моделям через миграции.

**Вопрос 2: Чем ModelForm отличается от обычной Form?**

Ответ: ModelForm привязана к конкретной модели. Автоматически генерирует поля формы на основе полей модели, выполняет валидацию с учётом типов данных модели, сохраняет данные через form.save() без необходимости вручную создавать объект.

**Вопрос 3: Как работает валидация в Django формах?**

Ответ: Валидация выполняется на нескольких уровнях: встроенная валидация полей (тип данных, обязательность), валидация через clean_методы для отдельных полей и clean() для всей формы. Ошибки добавляются в form.errors.